BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

PROGRAMMIERTECHNIK / 2026-2027

UML-Klassendiagramm

VIER GEWINNT · GYM EDITION

Objektorientierte Architektur des C++17-Lernprojekts

IST-MODELL
Game · Board · Player · Move

PROJEKTSTATUS
C++-Lernprojekt · kein GUI-Release

A3-DIAGRAMM · SEITE 2 | QUELLCODE-ABGLEICH | UML / OOP-LEGENDE

Das enthält diese Ausarbeitung

- **01 Gesamtübersicht**
  Alle vier Typen mit Sichtbarkeit, Methoden und drei Kompositionsbeziehungen.
- **02 Verantwortlichkeiten**
  Warum Game steuert, Board Regeln prüft und Player sowie Move Daten halten.
- **03 Beispielablauf**
  So arbeiten die Klassen bei einem gültigen Spielzug zusammen.
- **04 Abgleich und Abnahme**
  Welche Teile bereits im Code stehen und welche Erweiterungen geplant sind.

ZUM KLASSENDIAGRAMM SEITE 02 →

© 2026 Julian Krauß | JK-PT-2026-051-UML

DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

VIER GEWINNT / GYM EDITION

UML-KLASSENDIAGRAMM

Ist-Architektur des C++17-Lernprojekts · Stand: 22. Oktober 2026

BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

4 TYPEN · 3 BEZIEHUNGEN | C++17 / KONSOLENAPP | WERK-ID JK-PT-2026-051-UML

Game

STEUERUNG

ATTRIBUTE

```
- chosenPlayers_: int = 2
- chosenNames_: vector<string>
- players_: vector<Player>
- board_: Board
- history_: vector<Move>
- current_: int = 0
- active_: bool = false
- animation_: bool = true
- highscores_: map<string,int>
```

METHODEN

```
+ Game()
+ run(): void
- configurePlayers(): void
- configureNames(): void
- newGame(): void
- play(): void
- showHelp(): void
- showInfo(): void
- showHighscores(): void
- saveGame(path: string): bool
- loadGame(path: string): bool
- loadHighscores(): void
- saveHighscores(): bool
- addWin(name: string): void
```

1 — board_

Board

SPIELFELD

ATTRIBUTE

```
- rows_: int
- columns_: int
- grid_: vector<vector<char>>
```

METHODEN

```
- inside(row:int,col:int): bool
- countDirection(...): int
+ Board(playerCount: int = 2)
+ rows(): int
+ columns(): int
+ tokenCount(): int
+ nextFreeRow(column:int): int
+ drop(col:int,sym:char,row:int&): bool
+ undo(row:int,col:int): bool
+ hasWon(row:int,col:int,sym:char): bool
+ isFull(): bool
+ cells(): const grid&
+ print(previewCol:int=-1,...): void
+ animateDrop(col:int,sym:char,row:int): void
```

0..* history_ — 1

2..4 — players_

Player

SPIELERDATEN

ATTRIBUTE

```
- name_: string
- symbol_: char
```

METHODEN

```
+ Player(name:string,symbol:char)
+ getName(): const string&
+ getSymbol(): char
```

Move

«struct» ZUGDATEN

ATTRIBUTE

```
+ row: int
+ column: int
+ player: int
```

UML-Notation: + public - private Komposition / Teil gehört zur Game-Instanz

IST-STAND · keine grafische GUI-Klasse

© 2026 Julian Krauß · JK-CD-DE-2026-003 / JK-PT-2026-051-UML

A3 / skalierbare Vektorgrafik

02 / KLASSENVERANTWORTUNG

Vier Typen. Klare Aufgaben.

Die Klassennamen und Funktionen stammen aus den vorhandenen Header-Dateien des C++17-Lernprojekts.

Game / Orchestrierung

Verarbeitet Menübefehle, verwaltet die Spielerreihenfolge, hält die Partie aktiv und koordiniert Spielstände sowie Highscores.

Board / Spielregeln

Besitzt das zweidimensionale Raster; führt Fallbewegung, Vorschau, Rücknahme, Gewinnprüfung und Konsolenausgabe aus.

Player / Identität

Speichert Namen und Symbol eines Spielers. Die Siegzahl wird im aktuellen Stand nicht hier abgelegt.

Move / Historie

Ein einfacher struct mit Zeile, Spalte und Spielerindex. Game speichert Zug-Einträge in einem vector.

BEZIEHUNGEN / KOMPOSITION

Game besitzt genau ein Board-Objekt (1), je nach gewähltem Modus zwei bis vier Player-Objekte (2..4) und null bis beliebig viele Move-Einträge (0..*).

Weil die Objekte als Member bzw. als Elemente von Member-Vektoren zur Game-Instanz gehören, sind ausgefüllte Rauten (Komposition) ein passendes UML-Modell.

© 2026 Julian Krauß | JK-PT-2026-051-UML
DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

03 / SICHTBARKEIT / TYPEN

So liest du das Diagramm

Die Notation zeigt Attribute, Methoden, Sichtbarkeit und Beziehungen. UML bleibt vom Quellcode unterscheidbar.

- private

Interne Daten und Hilfsmethoden. Beispiel: - grid_: vector<vector<char>> ist nur innerhalb von Board zugreifbar.

+ public

Öffentliche Methoden bilden die nutzbare Schnittstelle. Beispiel: + drop(...) gibt bool zurück.

Komposition

Das ausgefüllte Rautensymbol steht auf der Besitzer-Seite, hier bei Game. Multiplikitäten: 1, 2..4 und 0..*.

«struct» Move

Move besitzt drei öffentliche Datenfelder. Es ist ein eigener Datentyp, kein zusätzlicher Spielcontroller.

BEISPIEL / C++-DEKLARATION → UML

```
private: Board board_;
public: void run();
// Game.h: vorhandene Deklarationen
```

```
UML:  - board_: Board
UML:  + run(): void
```

Die UML-Schreibweise nennt zuerst den Methodennamen und nach dem Doppelpunkt den Rückgabetyp. Das Beispiel entspricht den vorhandenen Game-Deklarationen.

© 2026 Julian Krauß | JK-PT-2026-051-UML

04 / ZUSAMMENSPIEL

Vom Menü bis zum Sieg

Beispiel für einen regulären Spielzug. Jede Stufe lässt sich im Sourcecode unmittelbar wiederfinden.

01

main.cpp

`Game game; game.run();`

Die Instanz entsteht. run() startet die Hauptmenü-Schleife.

02

Game::play()

`Eingabe und Zug validieren`

Game erkennt die Spalte und bestimmt den aktiven Spieler.

03

Board::drop()

`Kurzhantel im Raster ablegen`

Board berechnet die freie Zeile und aktualisiert genau eine Zelle.

04

Board::hasWon()

`Viererreihe überprüfen`

Board zählt in vier Richtungsachsen vom neuen Stein aus.

05

Game::addWin()

`Ergebnis dauerhaft erfassen`

Nur bei einem Sieg wird der Highscore erhöht und gespeichert.

Ohne Gewinn prüft Game auf Remis und wechselt weiter. Eine ungültige Eingabe verändert das Spielfeld nicht.

© 2026 Julian Krauß | JK-PT-2026-051-UML
DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

05 / NACHVOLLZIEHBARKEIT

Was ist im Code? Was ist geplant?

Für die Vorführung ist die Unterscheidung zwischen tatsächlichem C++-Code und optionalem Grafikdesign wichtig.

BAUSTEIN	QUELLCODE	ZUORDNUNG
Player	include/Player.h	Spielername, Symbol
Board	include/Board.h	Feld, Prüfungen, Animation
Game	include/Game.h	Menü, Spiel, Persistenz
Move	include/Game.h	Rückgängig-Historie
main()	src/main.cpp	Programmstart; keine Klasse

Wichtige Modellentscheidung

Die Schulaufgabe zeigt beispielhaft Player::wins. Im vorhandenen Lernprojekt steht die Siegzahl dagegen in Game::highscores_ als map<string,int>. Das Diagramm zeigt den tatsächlichen Code.

Gaming-UI = separates Zukunftsthema

Derzeit besitzt Board eine ASCII-Kurzhantelausgabe und Terminal-Fallanimation. Eine echte GUI, ein 3D-Renderer oder zusätzliche Fensterklassen sind noch nicht vorhanden.

ABNAHME / ERKLÄRUNG IM UNTERRICHT

Warum verändert Game nicht selbst das Raster? Wozu dient history_? Wann speichert addWin() den Highscore? Diese Fragen solltest du am Code erklären können.

© 2026 Julian Krauß | JK-PT-2026-051-UML